COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	October 5, 2026	**DATE SUBMITTED:**	October 5, 2026
NAME:	Jaime Luis M. Demain	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO. 10.1 **Graphs**

1. PROCEDURE OUTPUT

What procedures you perform?

Code

```
#include <iostream>

// Stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// Structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // Insert new nodes into adjacency list from given
graph
    adjNode* getAdjListNode(int value, int weight,
adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // Point new node to current
head
```

```
        return newNode;
    }

    int N; // Number of nodes in the graph

public:
    adjNode** head; // Adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // Allocate array of head pointers
        head = new adjNode*[N]();
        this->N = N;

        // Initialize head pointer for all vertices
        for (int i = 0; i < N; ++i) {
            head[i] = nullptr;
        }

        // Construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // Insert at the beginning
            adjNode* newNode = getAdjListNode(end_ver,
weight, head[start_ver]);
            head[start_ver] = newNode;
        }
    }

    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            adjNode* curr = head[i];
            while (curr != nullptr) {
```

```
                adjNode* next = curr->next;
                delete curr;
                curr = next;
            }
        }
        delete[] head;
    }
};

// Print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val << ", "
<< ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

int main() {
    // Graph edges array: (x, y, w) -> edge from x to y
with weight w
    graphEdge edges[] = {
        {0, 1, 2}, {0, 2, 4}, {1, 4, 3}, {2, 3, 2}, {3, 1,
4}, {4, 3, 3}
    };
    int N = 6; // Number of vertices in the graph
    int n = sizeof(edges) / sizeof(edges[0]);

    // Construct graph
    DiaGraph diagraph(edges, n, N);

    // Print adjacency list representation of graph
    std::cout << "Graph adjacency list " << std::endl
              << "(start_vertex, end_vertex, weight): " <<
std::endl;
    for (int i = 0; i < N; i++) {
```

```
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

Output

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)

*  Terminal will be reused by tasks, press any key to close it.
```

Observations

Each node's outgoing directed edges are stored using the adjacency list representation as a separate linked list. The reason for this is that the insert help function (getAdjListNode) inserts the incoming edges through the head pointer (newNode->next = head[start_ver]) such that the latter inserted edges appear first in the printed list (e.g., for vertex 0 we will get (0, 2, 4) before (0, 1, 2)). If a node has no outgoing directed edges (e.g., node 5), it produces an empty line.

Table 10-1. Graph Adjacency List

Code

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
```

```
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T>& e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T>& e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const {
        return V;
    }

    auto& edges() const {
        return edge_list;
    }

    void add_edge(Edge<T>&& e) {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 &&
e.dest <= V) {
            edge_list.emplace_back(e);
        } else {
            std::cerr << "Vertex out of bounds" <<
std::endl;
        }
    }

    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto& e : edge_list) {
```

```
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream& operator<<(std::ostream& os,
const Graph<U>& G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
std::ostream& operator<<(std::ostream& os, const Graph<T>&
G) {
    for (size_t i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto& e : edges) {
            os << "{" << e.dest << ": " << e.weight << "},
";
        }
        os << std::endl;
    }
    return os;
}

template <typename T>
auto depth_first_search(const Graph<T>& G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
```

```
    stack.push(1); // Assume DFS always starts from vertex
ID 1

    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();

        if (visited.find(current_vertex) == visited.end())
{
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e :
G.outgoing_edges(current_vertex)) {
                if (visited.find(e.dest) == visited.end())
{
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>>
edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
```

```
    for (auto& i : edges) {
        for (auto& j : i.second) {
            G.add_edge(Edge<T>{i.first, j.first,
j.second});
        }
    }
    return G;
}

template <typename T>
void test_DFS() {
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order) {
        std::cout << v << std::endl;
    }
}

int main() {
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Output

```
 *  Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe

1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
 *  Terminal will be reused by tasks, press any key to close it.
```

Observations

The depth first search algorithm uses an explicit stack structure implemented using std::stack with LIFO behavior. The outgoing edges from vertex 1 get pushed in that order (first 2, then 5), indicating that the vertex 5 gets pushed on the top of the stack and is the next to be popped off the stack. From the vertex 5, DFS pushes its non-visited neighbor vertices and explores deep down the graph in the branch $1 \rightarrow 5 \rightarrow 8 \rightarrow 6 \rightarrow 7 \rightarrow 3 \rightarrow 4 \rightarrow 2$.

Table 10-2. Depth-First Search (DFS)

Code

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge {
```

```
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T>& e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T>& e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const {
        return V;
    }

    auto& edges() const {
        return edge_list;
    }

    void add_edge(Edge<T>&& e) {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 &&
e.dest <= V) {
            edge_list.emplace_back(e);
        } else {
            std::cerr << "Vertex out of bounds" <<
std::endl;
        }
    }

    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
```

```
        for (auto& e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream& operator<<(std::ostream& os,
const Graph<U>& G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
std::ostream& operator<<(std::ostream& os, const Graph<T>&
G) {
    for (size_t i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto& e : edges) {
            os << "{" << e.dest << ": " << e.weight << "},
";
        }
        os << std::endl;
    }
    return os;
}

template <typename T>
auto breadth_first_search(const Graph<T>& G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
```

```
    std::set<size_t> visited;

    queue.push(1); // Assume BFS always starts from vertex
ID 1

    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();

        if (visited.find(current_vertex) == visited.end())
{
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e :
G.outgoing_edges(current_vertex)) {
                queue.push(e.dest);
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>>
edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};
```

```
    for (auto& i : edges) {
        for (auto& j : i.second) {
            G.add_edge(Edge<T>{i.first, j.first,
j.second});
        }
    }
    return G;
}

template <typename T>
void test_BFS() {
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order) {
        std::cout << v << std::endl;
    }
}

int main() {
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

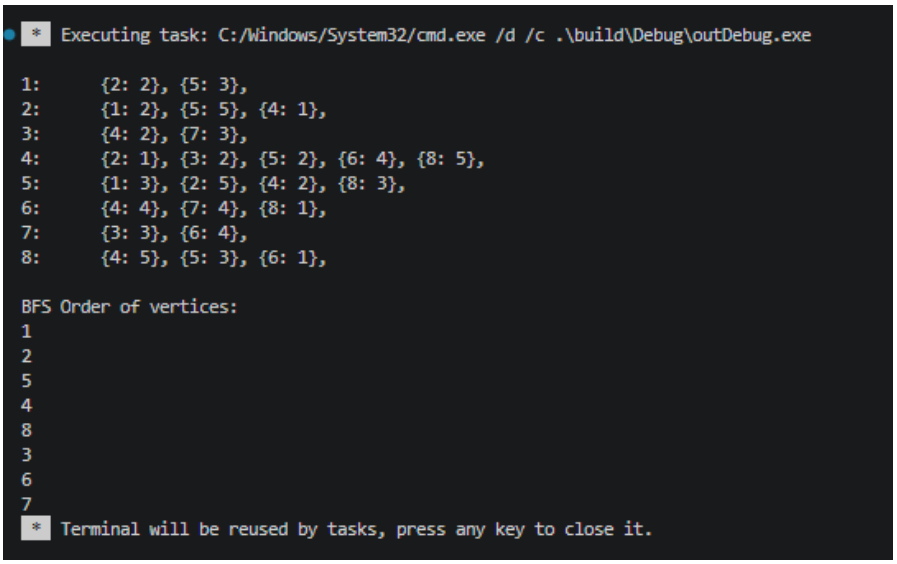
Output	
Observations	The BFS algorithm searches the graph node by node in a level order manner using a FIFO std::queue. Beginning from the root node 1, all the immediate neighboring nodes which are at one hop away from the source node (nodes 2 and 5) are queued up and searched first. It is only after searching the whole frontier that the BFS moves on to search for the second-hop-away neighbors (nodes 4 and 8), third-hop-away neighbors (nodes 3, 6, and 7).

Table 10-3. Breadth-First Search (BFS)

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

Answer:

The most helpful algorithm for this task is the **Depth-First Search (DFS)** algorithm.

Discussion:

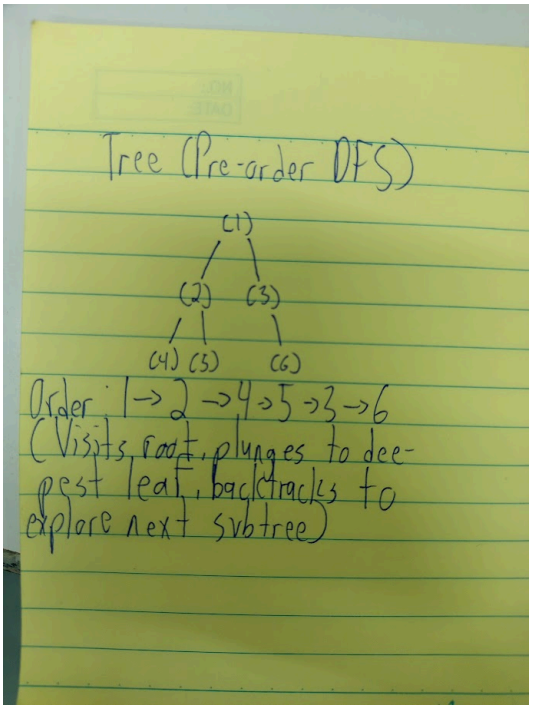
Description of the problem explains the basic operating principle of DFS that is exploring a path as far as possible in one branch only before going backwards. DFS works on the principle of LIFO stack, either iterative using std::stack or recursive using system call stack, to move down an unvisited path until the dead-end is found or the already visited vertex is encountered. Thereafter, it backtracks through the path to find the last vertex on the path with the unvisited adjacent branches and moves downwards from there. On the other hand, BFS visits all adjacent nodes level by level (concentrically).

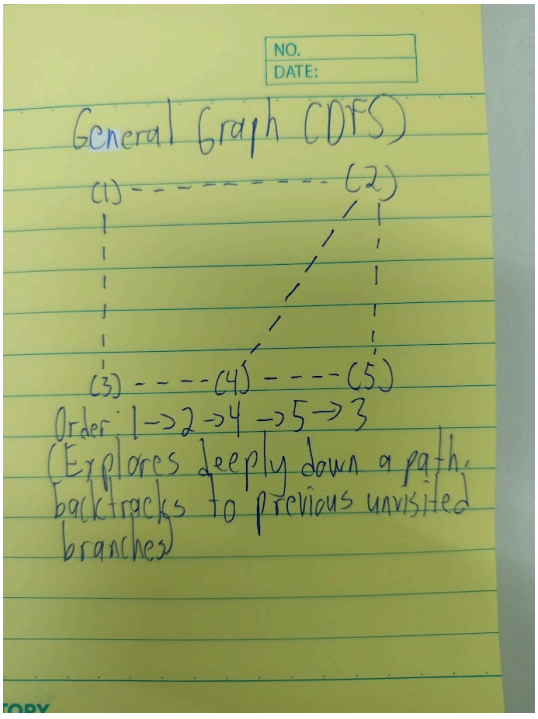
2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

Answer:

DFS's direct equivalent when considering tree traversal methods is **Pre-order Traversal** (and all other depth-first approaches like **In-order** and **Post-order Traversal**). This is because in the case of DFS, which starts at the root node, the order followed in the traversal is exactly the same as that in **Pre-order Traversal** (Root -> Left -> Right).

Graphical Comparison:





Pseudocode Examination:

- **Tree Pre-order Traversal:**

```
preOrder(node):
    if node is null: return
    visit(node)
    preOrder(node.left)
    preOrder(node.right)
```

- **Graph Depth-First Search:**

```
DFS(vertex v):
    mark v as visited
    visit(v)
    for each neighbor u adjacent to v:
        if u is not visited:
            DFS(u)
```

Comparison: Both algorithms use the same recursion structure. The only difference lies in the fact that while graphs need a visited list/array to avoid infinite loops, trees are acyclic by nature. All we need to do is compare whether a node is null or not.

- **Code Implementation (C++):**

```
// Tree Traversal (Pre-order DFS)
void treePreOrder(TreeNode* root) {
    if (root == nullptr) return;
    std::cout << root->val << " "; // Process Root
```

```
    treePreOrder(root->left);      // Dive Left
    treePreOrder(root->right);     // Dive Right
}

// Graph Traversal (DFS using visited tracker)
void graphDFS(int v, const std::vector<std::vector<int>>& adj, std::vector<bool>& visited) {
    visited[v] = true;             // Mark visited
    std::cout << v << " ";         // Process Vertex
    for (int neighbor : adj[v]) {  // Traverse all adjacent branches
        if (!visited[neighbor]) {
            graphDFS(neighbor, adj, visited); // Recurse / dive deeper
        }
    }
}
```

3. In the performed code, what data structure is used to implement the Breadth First Search?

Answer:

A **Queue** data structure (specifically, the standard library container **std::queue<size_t>**) is used to implement the Breadth-First Search.

Discussion:

A BFS approach to graph traversal entails visiting nodes level-wise based on a **FIFO principle**. The use of a queue ensures that all direct neighbors at the same distance level are visited before any vertices at more distant levels can be explored.

4. How many times can a node be visited in the BFS?

Answer:

A node is processed and added to the final visit order **exactly once.**

Discussion:

Despite the fact that an unvisited node might be detected or enqueued several times by the same node through different edges, the algorithm makes use of some kind of tracking mechanism such as std::set<size_t> visited or visited[] array. Guard condition:

```
if (visited.find(current_vertex) == visited.end()) {

    visited.insert(current_vertex);

    visit_order.push_back(current_vertex);

    ...

}
```

ensures that the vertex when popped will be considered visited and any further occurrence of the vertex will be skipped. This prevents any kind of duplicate operation and infinite loops in cyclic graphs.

Conclusion: This laboratory activity was particularly helpful in gaining in-depth knowledge regarding graph ADT implementations, their internal representations, and traversal algorithms in C++. As seen from the experiment, construction of the adjacency list structure using pointers-based linked nodes illustrated the ability to implement dynamic directed and weighted relationships with minimal memory consumption in comparison to dense adjacency matrices. Execution of Depth-First Search using an explicit stack and Breadth-First Search using a FIFO queue illustrated that the way data is stored influences the traversal process, i.e. DFS explores the path deeply at a time and then backtracks while BFS spreads out systematically across adjacent levels. The additional exercise served as further practice illustrating the necessity to use DFS for thorough exploration of a graph and backtracking on a map as well as proving that BFS should use visited sets in order to avoid repetitive computations and cycles within graphs. Generally, I performed well at implementing the templates, dealing with pointers, and monitoring the traversals; however, my weak point is connected with my poor understanding of graph edge-weight relaxations and visited-lookups overhead when moving towards such algorithms as

Dijkstra's or Prim's.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, Quillbot, etc.

Purpose of Use: **For code assistance in some parts of the procedure and forming a supplementary activity to understand its contents and improve grammar in forming an observation and conclusion.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI firstly to understand the concepts of the procedure part more and also improve the grammar of the observations of it. Second, to help me understand the supplementary activity. It's because I need to have more clarity on what each item of the supplementary is to answer it precisely. Lastly, I used AI to improve my grammar and tone in forming my conclusion.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.